# Imports

```
# PyTorch
import torch
print(f"Is CUDA available? {torch.cuda.is_available()}")
print(f"GPU Name: {torch.cuda.get_device_name(0)}")
print(f"CUDA Version: {torch.version.cuda}")
import torchvision
import torchvision.transforms as transforms
import torch.nn as nn
import torch.nn.functional as nnFunc
from torchviz import make_dot
# General
import numpy as np
import matplotlib.pyplot as plt
# Optimizer
import torch.optim as optim
# pretty print
from pprint import pprint
```

```
Is CUDA available? True
GPU Name: NVIDIA GeForce RTX 5050 Laptop GPU
CUDA Version: 13.0
```

## Dataset

This code uses the CIFAR-10 dataset. (https://www.cs.toronto.edu/~kriz/cifar.html)

## 1. Transform setting

The original images are in PIL (0 to 255 for each pixel). First we convert it to a tensor format (0 to 1). Then to make the data to be zero centering, we make the range to be between -1 to 1. If we use the following equation for each data, we can make it zero centered: \begin{equation} output = \frac{input - mean}{std} \end{equation} This is being done by setting the `mean` and `standard deviation` to be 0.5.

```
transform = transforms.Compose( [ transforms.ToTensor(),
                                  transforms.Normalize((0.5, 0.5,
0.5), (0.5, 0.5, 0.5))
                                ])
```

## 2. Making the training set

Now we load the actual dataset and make the training dataset.

1. Load CIFAR10 dataset (Download and prepare dataset)

    a. Put it in `./data` directory.
    b. Download it
    c. Do the specified transformation on the dataset
2. Create the data loader (pipelining settings)
    a. Brake it to batches with size of 32
    b. Shuffle the deck in every epoch
    c. Use 4 number of workers to make the training faster

```
batchSize = 256
numberOfWorkers = 8

trainSet = torchvision.datasets.CIFAR10(root='./data_CIFAR10',
                                        train=True,
                                        download=True,
                                        transform = transform)
trainLoader = torch.utils.data.DataLoader(trainSet,
                                          batch_size=batchSize,
                                          shuffle=True,
                                          num_workers=numberOfWorkers,
                                          pin_memory=True)
```

```
d:\Education\PhD\Courses\3_DeepLearning\Assignments\Repo\
202610ELC536501\.venv\Lib\site-packages\torchvision\datasets\
cifar.py:83: VisibleDeprecationWarning: dtype(): align should be
passed as Python or NumPy boolean but got `align=0`. Did you mean to
pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
  entry = pickle.load(f, encoding="latin1")
```

# 3. Making the test set

Now making the test set. The procedure is the same as making the training set with some minor modifications.

```
testSet = torchvision.datasets.CIFAR10(root='./data_CIFAR10',
                                       train=False, # Diff
                                       download=True,
                                       transform=transform)
testLoader = torch.utils.data.DataLoader(testSet,
                                         batch_size=batchSize,
                                         shuffle=False, # Diff
                                         num_workers=numberOfWorkers,
                                         pin_memory=True)
viz_batchSize = 4
vizLoader = torch.utils.data.DataLoader(testSet,
batch_size=viz_batchSize, shuffle=True)
```

## 4. The classes

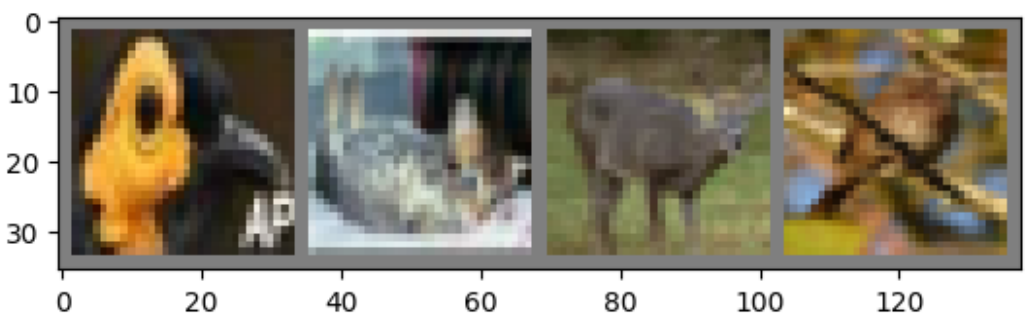
```
classes = ('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog',
'horse', 'ship', 'truck')
pprint(classes)
```

```
('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',
'ship', 'truck')
```

## DEUBG1

Showing some of the images from the dataset.

```
# functions to show an image
def imshow(img):
  img = img / 2 + 0.5 # unnormalize
  npimg = img.numpy()
  plt.imshow(np.transpose(npimg, (1, 2, 0)))
  plt.show()
# get some random training images
dataiter = iter(vizLoader)
images, labels = next(dataiter)
# show images
imshow(torchvision.utils.make_grid(images))
# print labels
pprint(' '.join(f'{classes[labels[j]]:5s}' for j in
range(viz_batchSize)))
```




```
'bird  cat   deer  bird '
```

## 5. Defining the CNN model

```
class Net(nn.Module):
  def __init__(self):
    super().__init__()
    self.conv1     = nn.Conv2d(3, 32, 3, padding=1)
    self.bn1       = nn.BatchNorm2d(32)
    self.conv2     = nn.Conv2d(32, 64, 3, padding=1)
```

```
    self.bn2      = nn.BatchNorm2d(64)
    self.pool     = nn.MaxPool2d(2, 2)
    self.lin1     = nn.Linear(64*8*8, 512)
    self.dropout  = nn.Dropout(0.3)
    self.lin2     = nn.Linear(512, 10)

  def forward(self, x):
    x = self.pool(nnFunc.relu(self.bn1(self.conv1(x))))
    x = self.pool(nnFunc.relu(self.bn2(self.conv2(x))))
    x = torch.flatten(x,1)
    x = nnFunc.relu(self.lin1(x))
    x = self.dropout(x)
    x = self.lin2(x)
    return x
```

## 6. Instansiating the CNN model

```
net = Net()
```

## 7. Move the model to GPU

```
device = torch.device('cuda:0' if torch.cuda.is_available() else
'cpu')
print(device)
```

```
cuda:0
```

```
net.to(device)
```

```
Net(
  (conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  (bn1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
  (lin1): Linear(in_features=4096, out_features=512, bias=True)
  (dropout): Dropout(p=0.3, inplace=False)
  (lin2): Linear(in_features=512, out_features=10, bias=True)
)
```

## 7. Degine loss function and optimizer

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters(), lr=0.001)
```

## 8. Traing the network

```
epochs = 30
printFrequency = len(trainLoader) // 4
for epoch in range(epochs):  # loop over the dataset multiple times
  running_loss = 0.0
  correct = 0
  total = 0
  for i, data in enumerate(trainLoader, 0):
    # get the inputs; data is a list of [inputs, labels]
    inputs, labels = data[0].to(device), data[1].to(device)
    # zero the parameter gradients
    optimizer.zero_grad()
    outputs = net(inputs)
    loss = criterion(outputs, labels)
    loss.backward()
    optimizer.step()
    # Accuracy Calculation
    _, predicted = torch.max(outputs.data, 1)
    total += labels.size(0)
    correct += (predicted == labels).sum().item()
    # Runnig loss
    running_loss += loss.item()
    # Printing
    if i % printFrequency == (printFrequency - 1):    # print every
2000 mini-batches
      accuracy = 100 * correct / total
      print(f'Epoch {epoch + 1} | Batch {i + 1}/{len(trainLoader)} |
loss: {running_loss / printFrequency:.3f} | acc: {accuracy:.2f}%')
      # Reset all the counters
      running_loss = 0.0
      correct = 0
      total = 0

print('Finished Training')
```

```
Epoch 1 | Batch 49/196 | loss: 2.108 | acc: 29.55%
Epoch 1 | Batch 98/196 | loss: 1.546 | acc: 44.20%
Epoch 1 | Batch 147/196 | loss: 1.368 | acc: 50.37%
Epoch 1 | Batch 196/196 | loss: 1.274 | acc: 54.25%
Epoch 2 | Batch 49/196 | loss: 1.191 | acc: 57.40%
Epoch 2 | Batch 98/196 | loss: 1.108 | acc: 60.89%
Epoch 2 | Batch 147/196 | loss: 1.089 | acc: 61.25%
Epoch 2 | Batch 196/196 | loss: 1.036 | acc: 62.94%
Epoch 3 | Batch 49/196 | loss: 0.984 | acc: 64.88%
Epoch 3 | Batch 98/196 | loss: 0.944 | acc: 66.42%
Epoch 3 | Batch 147/196 | loss: 0.934 | acc: 67.17%
Epoch 3 | Batch 196/196 | loss: 0.939 | acc: 66.96%
Epoch 4 | Batch 49/196 | loss: 0.866 | acc: 69.38%
Epoch 4 | Batch 98/196 | loss: 0.855 | acc: 69.83%
```

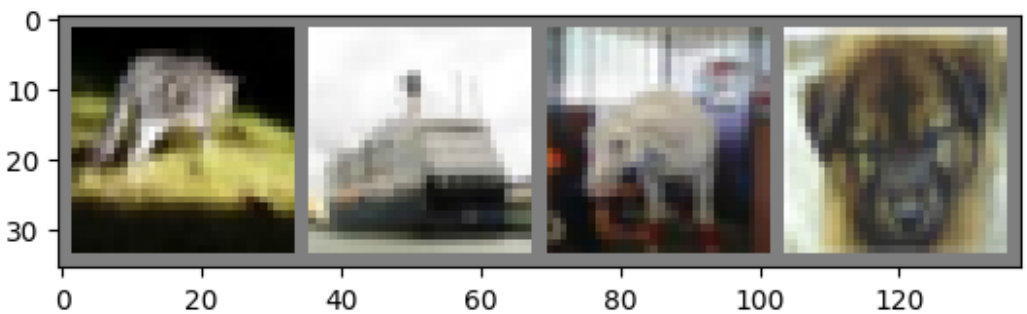
```
Epoch 4 | Batch 147/196 | loss: 0.859 | acc: 69.98%
Epoch 4 | Batch 196/196 | loss: 0.850 | acc: 70.21%
Epoch 5 | Batch 49/196 | loss: 0.792 | acc: 72.16%
Epoch 5 | Batch 98/196 | loss: 0.793 | acc: 72.03%
Epoch 5 | Batch 147/196 | loss: 0.800 | acc: 71.61%
Epoch 5 | Batch 196/196 | loss: 0.774 | acc: 73.01%
Epoch 6 | Batch 49/196 | loss: 0.734 | acc: 73.95%
Epoch 6 | Batch 98/196 | loss: 0.737 | acc: 73.87%
Epoch 6 | Batch 147/196 | loss: 0.759 | acc: 73.01%
Epoch 6 | Batch 196/196 | loss: 0.753 | acc: 73.63%
Epoch 7 | Batch 49/196 | loss: 0.672 | acc: 76.31%
Epoch 7 | Batch 98/196 | loss: 0.680 | acc: 76.15%
Epoch 7 | Batch 147/196 | loss: 0.686 | acc: 75.44%
Epoch 7 | Batch 196/196 | loss: 0.696 | acc: 75.71%
Epoch 8 | Batch 49/196 | loss: 0.635 | acc: 77.82%
Epoch 8 | Batch 98/196 | loss: 0.627 | acc: 78.51%
Epoch 8 | Batch 147/196 | loss: 0.645 | acc: 77.46%
Epoch 8 | Batch 196/196 | loss: 0.641 | acc: 77.26%
Epoch 9 | Batch 49/196 | loss: 0.570 | acc: 79.91%
Epoch 9 | Batch 98/196 | loss: 0.591 | acc: 79.29%
Epoch 9 | Batch 147/196 | loss: 0.602 | acc: 79.18%
Epoch 9 | Batch 196/196 | loss: 0.621 | acc: 78.15%
Epoch 10 | Batch 49/196 | loss: 0.563 | acc: 80.01%
Epoch 10 | Batch 98/196 | loss: 0.543 | acc: 80.76%
Epoch 10 | Batch 147/196 | loss: 0.572 | acc: 79.80%
Epoch 10 | Batch 196/196 | loss: 0.593 | acc: 78.98%
Epoch 11 | Batch 49/196 | loss: 0.516 | acc: 81.82%
Epoch 11 | Batch 98/196 | loss: 0.513 | acc: 81.78%
Epoch 11 | Batch 147/196 | loss: 0.530 | acc: 81.08%
Epoch 11 | Batch 196/196 | loss: 0.532 | acc: 81.11%
Epoch 12 | Batch 49/196 | loss: 0.472 | acc: 83.22%
Epoch 12 | Batch 98/196 | loss: 0.485 | acc: 82.80%
Epoch 12 | Batch 147/196 | loss: 0.476 | acc: 82.89%
Epoch 12 | Batch 196/196 | loss: 0.504 | acc: 82.26%
Epoch 13 | Batch 49/196 | loss: 0.443 | acc: 83.86%
Epoch 13 | Batch 98/196 | loss: 0.452 | acc: 84.08%
Epoch 13 | Batch 147/196 | loss: 0.457 | acc: 83.79%
Epoch 13 | Batch 196/196 | loss: 0.461 | acc: 83.74%
Epoch 14 | Batch 49/196 | loss: 0.410 | acc: 85.30%
Epoch 14 | Batch 98/196 | loss: 0.421 | acc: 85.37%
Epoch 14 | Batch 147/196 | loss: 0.431 | acc: 84.91%
Epoch 14 | Batch 196/196 | loss: 0.440 | acc: 84.18%
Epoch 15 | Batch 49/196 | loss: 0.379 | acc: 86.29%
Epoch 15 | Batch 98/196 | loss: 0.393 | acc: 86.36%
Epoch 15 | Batch 147/196 | loss: 0.417 | acc: 84.67%
Epoch 15 | Batch 196/196 | loss: 0.408 | acc: 85.18%
Epoch 16 | Batch 49/196 | loss: 0.363 | acc: 86.93%
Epoch 16 | Batch 98/196 | loss: 0.368 | acc: 86.68%
Epoch 16 | Batch 147/196 | loss: 0.373 | acc: 86.62%
```

```
Epoch 16 | Batch 196/196 | loss: 0.386 | acc: 85.93%
Epoch 17 | Batch 49/196 | loss: 0.339 | acc: 87.75%
Epoch 17 | Batch 98/196 | loss: 0.333 | acc: 88.05%
Epoch 17 | Batch 147/196 | loss: 0.352 | acc: 87.34%
Epoch 17 | Batch 196/196 | loss: 0.375 | acc: 86.33%
Epoch 18 | Batch 49/196 | loss: 0.316 | acc: 88.64%
Epoch 18 | Batch 98/196 | loss: 0.314 | acc: 88.90%
Epoch 18 | Batch 147/196 | loss: 0.338 | acc: 87.85%
Epoch 18 | Batch 196/196 | loss: 0.338 | acc: 87.76%
Epoch 19 | Batch 49/196 | loss: 0.271 | acc: 90.27%
Epoch 19 | Batch 98/196 | loss: 0.297 | acc: 89.13%
Epoch 19 | Batch 147/196 | loss: 0.313 | acc: 88.76%
Epoch 19 | Batch 196/196 | loss: 0.311 | acc: 88.75%
Epoch 20 | Batch 49/196 | loss: 0.276 | acc: 90.24%
Epoch 20 | Batch 98/196 | loss: 0.283 | acc: 89.87%
Epoch 20 | Batch 147/196 | loss: 0.276 | acc: 90.05%
Epoch 20 | Batch 196/196 | loss: 0.304 | acc: 88.86%
Epoch 21 | Batch 49/196 | loss: 0.263 | acc: 90.53%
Epoch 21 | Batch 98/196 | loss: 0.267 | acc: 90.47%
Epoch 21 | Batch 147/196 | loss: 0.276 | acc: 90.22%
Epoch 21 | Batch 196/196 | loss: 0.284 | acc: 89.98%
Epoch 22 | Batch 49/196 | loss: 0.232 | acc: 91.71%
Epoch 22 | Batch 98/196 | loss: 0.244 | acc: 91.42%
Epoch 22 | Batch 147/196 | loss: 0.262 | acc: 90.61%
Epoch 22 | Batch 196/196 | loss: 0.270 | acc: 90.54%
Epoch 23 | Batch 49/196 | loss: 0.225 | acc: 91.92%
Epoch 23 | Batch 98/196 | loss: 0.224 | acc: 91.60%
Epoch 23 | Batch 147/196 | loss: 0.238 | acc: 91.53%
Epoch 23 | Batch 196/196 | loss: 0.248 | acc: 91.03%
Epoch 24 | Batch 49/196 | loss: 0.230 | acc: 91.61%
Epoch 24 | Batch 98/196 | loss: 0.218 | acc: 92.16%
Epoch 24 | Batch 147/196 | loss: 0.219 | acc: 91.88%
Epoch 24 | Batch 196/196 | loss: 0.247 | acc: 91.13%
Epoch 25 | Batch 49/196 | loss: 0.225 | acc: 91.73%
Epoch 25 | Batch 98/196 | loss: 0.215 | acc: 92.53%
Epoch 25 | Batch 147/196 | loss: 0.229 | acc: 91.61%
Epoch 25 | Batch 196/196 | loss: 0.227 | acc: 91.66%
Epoch 26 | Batch 49/196 | loss: 0.202 | acc: 92.68%
Epoch 26 | Batch 98/196 | loss: 0.189 | acc: 93.18%
Epoch 26 | Batch 147/196 | loss: 0.218 | acc: 92.20%
Epoch 26 | Batch 196/196 | loss: 0.221 | acc: 92.29%
Epoch 27 | Batch 49/196 | loss: 0.186 | acc: 93.38%
Epoch 27 | Batch 98/196 | loss: 0.177 | acc: 93.66%
Epoch 27 | Batch 147/196 | loss: 0.197 | acc: 93.10%
Epoch 27 | Batch 196/196 | loss: 0.203 | acc: 92.75%
Epoch 28 | Batch 49/196 | loss: 0.174 | acc: 93.65%
Epoch 28 | Batch 98/196 | loss: 0.185 | acc: 93.33%
Epoch 28 | Batch 147/196 | loss: 0.185 | acc: 93.42%
Epoch 28 | Batch 196/196 | loss: 0.194 | acc: 92.82%
```

```
Epoch 29 | Batch 49/196 | loss: 0.165 | acc: 93.89%
Epoch 29 | Batch 98/196 | loss: 0.165 | acc: 94.09%
Epoch 29 | Batch 147/196 | loss: 0.172 | acc: 93.89%
Epoch 29 | Batch 196/196 | loss: 0.189 | acc: 93.15%
Epoch 30 | Batch 49/196 | loss: 0.152 | acc: 94.80%
Epoch 30 | Batch 98/196 | loss: 0.147 | acc: 94.75%
Epoch 30 | Batch 147/196 | loss: 0.159 | acc: 94.28%
Epoch 30 | Batch 196/196 | loss: 0.184 | acc: 93.48%
Finished Training
```

## 9. Test the network on the test data

```
dataIter = iter(vizLoader)
images, labels = next(dataIter)
# print images
imshow(torchvision.utils.make_grid(images))
print('GroundTruth: ', ' '.join(f'{classes[labels[j]]:5s}' for j in
range(viz_batchSize)))
```




```
GroundTruth:  cat   ship  horse dog

# Move the data to GPU
images, labels = images.to(device), labels.to(device)

# Now you can run it
outputs = net(images)
_, predicted = torch.max(outputs, 1)

# IMPORTANT: To print, move back to CPU
# Printing needs the labels to be on the CPU so Python can read them
print('Predicted: ', ' '.join(f'{classes[predicted[j].item()]:5s}' for
j in range(len(predicted))))

Predicted:  horse ship  cat   dog
```

## 10. Test the network on the whole dataset

```
test_loss = 0.0
correct = 0
total = 0

with torch.no_grad():
    for data in testLoader:
        images, labels = data[0].to(device), data[1].to(device)

        outputs = net(images)

        # 1. Calculate the loss for this batch
        loss = criterion(outputs, labels)
        test_loss += loss.item()

        # 2. Calculate accuracy as before
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

# Calculate the average loss over all batches
avg_test_loss = test_loss / len(testLoader)

print(f'Test Loss: {avg_test_loss:.4f}')
print(f'Test Accuracy: {100 * correct / total:.2f}%')
```

```
Test Loss: 1.2133
Test Accuracy: 71.77%
```

## 11. Check performance within each class

```
# prepare to count predictions for each class
correct_pred = {classname: 0 for classname in classes}
total_pred = {classname: 0 for classname in classes}

# again no gradients needed
with torch.no_grad():
    for data in testLoader:
        images, labels = data[0].to(device), data[1].to(device)
        outputs = net(images)
        _, predictions = torch.max(outputs, 1)
        # collect the correct predictions for each class
        for label, prediction in zip(labels, predictions):
            if label == prediction:
                correct_pred[classes[label]] += 1
            total_pred[classes[label]] += 1

# print accuracy for each class
for classname, correct_count in correct_pred.items():
```

```
    accuracy = 100 * float(correct_count) / total_pred[classname]
    print(f'Accuracy for class: {classname:5s} is {accuracy:.1f} %')
```

```
Accuracy for class: plane is 73.6 %
Accuracy for class: car   is 86.0 %
Accuracy for class: bird  is 57.4 %
Accuracy for class: cat   is 56.1 %
Accuracy for class: deer  is 69.1 %
Accuracy for class: dog   is 68.5 %
Accuracy for class: frog  is 72.4 %
Accuracy for class: horse is 79.0 %
Accuracy for class: ship  is 80.2 %
Accuracy for class: truck is 78.9 %
```

## Graphic

```
# Create a dummy input that matches your image size (1 image, 3
channels, 32x32)
dummy_input = torch.randn(1, 3, 32, 32).to(device)

# Run the input through your net
out = net(dummy_input)

# Generate the visualization
dot = make_dot(out, params=dict(net.named_parameters()))
dot.format = 'png'
dot.render("neural_network_graph")
dot
```

```
torch.onnx.export(net.cpu(), torch.randn(1, 3, 32, 32), "model.onnx")
```

```
C:\Users\kaveh\AppData\Local\Temp\ipykernel_27940\2844925536.py:1:
UserWarning: Exporting a model while it is in training mode. Please
ensure that this is intended, as it may lead to different behavior
during inference. Calling model.eval() before export is recommended.
  torch.onnx.export(net.cpu(), torch.randn(1, 3, 32, 32),
"model.onnx")
```

```
W0315 02:25:17.998000 27940 Lib\site-packages\torch\onnx\_internal\
exporter\_schemas.py:455] Missing annotation for parameter 'input'
from (input, boxes, output_size: 'Sequence[int]', spatial_scale:
'float' = 1.0, sampling_ratio: 'int' = -1, aligned: 'bool' = False).
Treating as an Input.
W0315 02:25:18.000000 27940 Lib\site-packages\torch\onnx\_internal\
exporter\_schemas.py:455] Missing annotation for parameter 'boxes'
from (input, boxes, output_size: 'Sequence[int]', spatial_scale:
'float' = 1.0, sampling_ratio: 'int' = -1, aligned: 'bool' = False).
Treating as an Input.
W0315 02:25:18.002000 27940 Lib\site-packages\torch\onnx\_internal\
exporter\_schemas.py:455] Missing annotation for parameter 'input'
from (input, boxes, output_size: 'Sequence[int]', spatial_scale:
'float' = 1.0). Treating as an Input.
W0315 02:25:18.003000 27940 Lib\site-packages\torch\onnx\_internal\
exporter\_schemas.py:455] Missing annotation for parameter 'boxes'
from (input, boxes, output_size: 'Sequence[int]', spatial_scale:
'float' = 1.0). Treating as an Input.

[torch.onnx] Obtain model graph for `Net([...]` with
`torch.export.export(..., strict=False)`...
[torch.onnx] Obtain model graph for `Net([...]` with
`torch.export.export(..., strict=False)`... □
[torch.onnx] Run decomposition...
[torch.onnx] Run decomposition... □
[torch.onnx] Translate the graph into ONNX...
[torch.onnx] Translate the graph into ONNX... □
Applied 2 of general pattern rewrite rules.

C:\Python312\Lib\copyreg.py:99: FutureWarning: `isinstance(treespec,
LeafSpec)` is deprecated, use `isinstance(treespec, TreeSpec) and
treespec.is_leaf()` instead.
  return cls.__new__(cls, *args)

ONNXProgram(
    model=
        <
            ir_version=10,
            opset_imports={'': 20},
            producer_name='pytorch',
            producer_version='2.10.0+cu130',
            domain=None,
            model_version=None,
        >
        graph(
            name=main_graph,
            inputs=(
                %"x"<FLOAT,[1,3,32,32]>
            ),
            outputs=(
```

```
                %"linear_1"<FLOAT,[1,10]>
            ),
            initializers=(
                %"conv1.weight"<FLOAT,[32,3,3,3]>{Tensor(...)},
                %"conv1.bias"<FLOAT,[32]>{Tensor(...)},
                %"conv2.weight"<FLOAT,[64,32,3,3]>{Tensor(...)},
                %"conv2.bias"<FLOAT,[64]>{Tensor(...)},
                %"lin1.weight"<FLOAT,[512,4096]>{TorchTensor(...)},
                %"lin1.bias"<FLOAT,[512]>{TorchTensor(...)},
                %"lin2.weight"<FLOAT,[10,512]>{TorchTensor(...)},
                %"lin2.bias"<FLOAT,[10]>{TorchTensor<FLOAT,
[10]>(Parameter containing: tensor([-0.0277, -0.2529, -0.1307,
0.1025,  0.0726,  0.0473, -0.0100,  0.0608, -0.1054,  0.1673],
requires_grad=True), name='lin2.bias')},
                %"val_16"<INT64,[2]>{Tensor<INT64,[2]>(array([   1,
4096]), name='val_16')},
                %"val_17"<FLOAT,[]>{Tensor<FLOAT,[]>(array(0.3,
dtype=float32), name='val_17')},
                %"val_18"<BOOL,[]>{Tensor<BOOL,[]>(array(True),
name='val_18')}
            ),
        ) {
             0 |  # node_Conv_17
                  %"getitem"<FLOAT,[1,32,32,32]> ← ::Conv(%"x",
%"conv1.weight"{...}, %"conv1.bias"{...}) {group=1, auto_pad='NOTSET',
dilations=(1, 1), strides=(1, 1), pads=(1, 1, 1, 1)}
             1 |  # node_relu
                  %"relu"<FLOAT,[1,32,32,32]> ← ::Relu(%"getitem")
             2 |  # node_max_pool2d
                  %"max_pool2d"<FLOAT,[1,32,16,16]>
← ::MaxPool(%"relu") {kernel_shape=(2, 2), strides=(2, 2),
dilations=(1, 1), auto_pad='NOTSET', pads=(0, 0, 0, 0), ceil_mode=0,
storage_order=0}
             3 |  # node_Conv_18
                  %"getitem_5"<FLOAT,[1,64,16,16]>
← ::Conv(%"max_pool2d", %"conv2.weight"{...}, %"conv2.bias"{...})
{group=1, auto_pad='NOTSET', dilations=(1, 1), strides=(1, 1),
pads=(1, 1, 1, 1)}
             4 |  # node_relu_1
                  %"relu_1"<FLOAT,[1,64,16,16]> ← ::Relu(%"getitem_5")
             5 |  # node_max_pool2d_1
                  %"max_pool2d_1"<FLOAT,[1,64,8,8]>
← ::MaxPool(%"relu_1") {kernel_shape=(2, 2), strides=(2, 2),
dilations=(1, 1), auto_pad='NOTSET', pads=(0, 0, 0, 0), ceil_mode=0,
storage_order=0}
             6 |  # node_view
                  %"view"<FLOAT,[1,4096]> ← ::Reshape(%"max_pool2d_1",
%"val_16"{[1, 4096]}) {allowzero=1}
             7 |  # node_linear
```

```
                  %"linear"<FLOAT,[1,512]> ← ::Gemm(%"view",
%"lin1.weight"{...}, %"lin1.bias"{...}) {transA=0, transB=1,
alpha=1.0, beta=1.0}
             8 |  # node_relu_2
                  %"relu_2"<FLOAT,[1,512]> ← ::Relu(%"linear")
             9 |  # node_native_dropout__1
                  %"getitem_10"<FLOAT,[1,512]> ← ::Dropout(%"relu_2",
%"val_17"{0.30000001192092896}, %"val_18"{True})
            10 |  # node_linear_1
                  %"linear_1"<FLOAT,[1,10]> ← ::Gemm(%"getitem_10",
%"lin2.weight"{...}, %"lin2.bias"{[-0.02768433839082718, -
0.2528810203075409, -0.13068464398384094, 0.1025228425860405,
0.07255247235298157, 0.04730239883065224, -0.010006977245211601,
0.06084134802222252, -0.1054290309548378, 0.1672801971435547]})
{transA=0, transB=1, alpha=1.0, beta=1.0}
            return %"linear_1"<FLOAT,[1,10]>
        }


    ,
    exported_program=
        ExportedProgram:
            class GraphModule(torch.nn.Module):
                def forward(self, p_conv1_weight: "f32[32, 3, 3, 3]",
p_conv1_bias: "f32[32]", p_bn1_weight: "f32[32]", p_bn1_bias:
"f32[32]", p_conv2_weight: "f32[64, 32, 3, 3]", p_conv2_bias:
"f32[64]", p_bn2_weight: "f32[64]", p_bn2_bias: "f32[64]",
p_lin1_weight: "f32[512, 4096]", p_lin1_bias: "f32[512]",
p_lin2_weight: "f32[10, 512]", p_lin2_bias: "f32[10]",
b_bn1_running_mean: "f32[32]", b_bn1_running_var: "f32[32]",
b_bn1_num_batches_tracked: "i64[]", b_bn2_running_mean: "f32[64]",
b_bn2_running_var: "f32[64]", b_bn2_num_batches_tracked: "i64[]", x:
"f32[1, 3, 32, 32]"):
                    # File: d:\Education\PhD\Courses\3_DeepLearning\
Assignments\Repo\202610ELC536501\.venv\Lib\site-packages\torch\nn\
modules\conv.py:553 in forward, code: return self._conv_forward(input,
self.weight, self.bias)
                    conv2d: "f32[1, 32, 32, 32]" =
torch.ops.aten.conv2d.default(x, p_conv1_weight, p_conv1_bias, [1, 1],
[1, 1]);  x = p_conv1_weight = p_conv1_bias = None

                    # File: d:\Education\PhD\Courses\3_DeepLearning\
Assignments\Repo\202610ELC536501\.venv\Lib\site-packages\torch\nn\
modules\batchnorm.py:174 in forward, code:
self.num_batches_tracked.add_(1)  # type: ignore[has-type]
                    add: "i64[]" =
torch.ops.aten.add.Tensor(b_bn1_num_batches_tracked, 1);
b_bn1_num_batches_tracked = None
```

```
                    # File: d:\Education\PhD\Courses\3_DeepLearning\
Assignments\Repo\202610ELC536501\.venv\Lib\site-packages\torch\nn\
modules\batchnorm.py:194 in forward, code: return F.batch_norm(
                    _native_batch_norm_legit_functional =
torch.ops.aten._native_batch_norm_legit_functional.default(conv2d,
p_bn1_weight, p_bn1_bias, b_bn1_running_mean, b_bn1_running_var, True,
0.1, 1e-05);  conv2d = p_bn1_weight = p_bn1_bias = b_bn1_running_mean
= b_bn1_running_var = None
                    getitem: "f32[1, 32, 32, 32]" =
_native_batch_norm_legit_functional[0]
                    getitem_3: "f32[32]" =
_native_batch_norm_legit_functional[3]
                    getitem_4: "f32[32]" =
_native_batch_norm_legit_functional[4];
_native_batch_norm_legit_functional = None

                    # File: C:\Users\kaveh\AppData\Local\Temp\
ipykernel_27940\3924601976.py:14 in forward, code: x =
self.pool(nnFunc.relu(self.bn1(self.conv1(x))))
                    relu: "f32[1, 32, 32, 32]" =
torch.ops.aten.relu.default(getitem);  getitem = None

                    # File: d:\Education\PhD\Courses\3_DeepLearning\
Assignments\Repo\202610ELC536501\.venv\Lib\site-packages\torch\nn\
modules\pooling.py:224 in forward, code: return F.max_pool2d(
                    max_pool2d: "f32[1, 32, 16, 16]" =
torch.ops.aten.max_pool2d.default(relu, [2, 2], [2, 2]);  relu = None

                    # File: d:\Education\PhD\Courses\3_DeepLearning\
Assignments\Repo\202610ELC536501\.venv\Lib\site-packages\torch\nn\
modules\conv.py:553 in forward, code: return self._conv_forward(input,
self.weight, self.bias)
                    conv2d_1: "f32[1, 64, 16, 16]" =
torch.ops.aten.conv2d.default(max_pool2d, p_conv2_weight,
p_conv2_bias, [1, 1], [1, 1]);  max_pool2d = p_conv2_weight =
p_conv2_bias = None

                    # File: d:\Education\PhD\Courses\3_DeepLearning\
Assignments\Repo\202610ELC536501\.venv\Lib\site-packages\torch\nn\
modules\batchnorm.py:174 in forward, code:
self.num_batches_tracked.add_(1)  # type: ignore[has-type]
                    add_1: "i64[]" =
torch.ops.aten.add.Tensor(b_bn2_num_batches_tracked, 1);
b_bn2_num_batches_tracked = None

                    # File: d:\Education\PhD\Courses\3_DeepLearning\
Assignments\Repo\202610ELC536501\.venv\Lib\site-packages\torch\nn\
modules\batchnorm.py:194 in forward, code: return F.batch_norm(
                    _native_batch_norm_legit_functional_1 =
```

```
torch.ops.aten._native_batch_norm_legit_functional.default(conv2d_1,
p_bn2_weight, p_bn2_bias, b_bn2_running_mean, b_bn2_running_var, True,
0.1, 1e-05);  conv2d_1 = p_bn2_weight = p_bn2_bias =
b_bn2_running_mean = b_bn2_running_var = None
                    getitem_5: "f32[1, 64, 16, 16]" =
_native_batch_norm_legit_functional_1[0]
                    getitem_8: "f32[64]" =
_native_batch_norm_legit_functional_1[3]
                    getitem_9: "f32[64]" =
_native_batch_norm_legit_functional_1[4];
_native_batch_norm_legit_functional_1 = None

                    # File: C:\Users\kaveh\AppData\Local\Temp\
ipykernel_27940\3924601976.py:15 in forward, code: x =
self.pool(nnFunc.relu(self.bn2(self.conv2(x))))
                    relu_1: "f32[1, 64, 16, 16]" =
torch.ops.aten.relu.default(getitem_5);  getitem_5 = None

                    # File: d:\Education\PhD\Courses\3_DeepLearning\
Assignments\Repo\202610ELC536501\.venv\Lib\site-packages\torch\nn\
modules\pooling.py:224 in forward, code: return F.max_pool2d(
                    max_pool2d_1: "f32[1, 64, 8, 8]" =
torch.ops.aten.max_pool2d.default(relu_1, [2, 2], [2, 2]);  relu_1 =
None

                    # File: C:\Users\kaveh\AppData\Local\Temp\
ipykernel_27940\3924601976.py:16 in forward, code: x =
torch.flatten(x,1)
                    view: "f32[1, 4096]" =
torch.ops.aten.view.default(max_pool2d_1, [1, 4096]);  max_pool2d_1 =
None

                    # File: d:\Education\PhD\Courses\3_DeepLearning\
Assignments\Repo\202610ELC536501\.venv\Lib\site-packages\torch\nn\
modules\linear.py:134 in forward, code: return F.linear(input,
self.weight, self.bias)
                    linear: "f32[1, 512]" =
torch.ops.aten.linear.default(view, p_lin1_weight, p_lin1_bias);  view
= p_lin1_weight = p_lin1_bias = None

                    # File: C:\Users\kaveh\AppData\Local\Temp\
ipykernel_27940\3924601976.py:17 in forward, code: x =
nnFunc.relu(self.lin1(x))
                    relu_2: "f32[1, 512]" =
torch.ops.aten.relu.default(linear);  linear = None

                    # File: d:\Education\PhD\Courses\3_DeepLearning\
Assignments\Repo\202610ELC536501\.venv\Lib\site-packages\torch\nn\
modules\dropout.py:73 in forward, code: return F.dropout(input,
```

```
self.p, self.training, self.inplace)
                    native_dropout =
torch.ops.aten.native_dropout.default(relu_2, 0.3, True);  relu_2 =
None
                    getitem_10: "f32[1, 512]" = native_dropout[0];
native_dropout = None

                    # File: d:\Education\PhD\Courses\3_DeepLearning\
Assignments\Repo\202610ELC536501\.venv\Lib\site-packages\torch\nn\
modules\linear.py:134 in forward, code: return F.linear(input,
self.weight, self.bias)
                    linear_1: "f32[1, 10]" =
torch.ops.aten.linear.default(getitem_10, p_lin2_weight, p_lin2_bias);
getitem_10 = p_lin2_weight = p_lin2_bias = None
                    return (getitem_3, getitem_4, add, getitem_8,
getitem_9, add_1, linear_1)

        Graph signature:
            # inputs
            p_conv1_weight: PARAMETER target='conv1.weight'
            p_conv1_bias: PARAMETER target='conv1.bias'
            p_bn1_weight: PARAMETER target='bn1.weight'
            p_bn1_bias: PARAMETER target='bn1.bias'
            p_conv2_weight: PARAMETER target='conv2.weight'
            p_conv2_bias: PARAMETER target='conv2.bias'
            p_bn2_weight: PARAMETER target='bn2.weight'
            p_bn2_bias: PARAMETER target='bn2.bias'
            p_lin1_weight: PARAMETER target='lin1.weight'
            p_lin1_bias: PARAMETER target='lin1.bias'
            p_lin2_weight: PARAMETER target='lin2.weight'
            p_lin2_bias: PARAMETER target='lin2.bias'
            b_bn1_running_mean: BUFFER target='bn1.running_mean'
persistent=True
            b_bn1_running_var: BUFFER target='bn1.running_var'
persistent=True
            b_bn1_num_batches_tracked: BUFFER
target='bn1.num_batches_tracked' persistent=True
            b_bn2_running_mean: BUFFER target='bn2.running_mean'
persistent=True
            b_bn2_running_var: BUFFER target='bn2.running_var'
persistent=True
            b_bn2_num_batches_tracked: BUFFER
target='bn2.num_batches_tracked' persistent=True
            x: USER_INPUT

            # outputs
            getitem_3: BUFFER_MUTATION target='bn1.running_mean'
            getitem_4: BUFFER_MUTATION target='bn1.running_var'
            add: BUFFER_MUTATION target='bn1.num_batches_tracked'
```

```
            getitem_8: BUFFER_MUTATION target='bn2.running_mean'
            getitem_9: BUFFER_MUTATION target='bn2.running_var'
            add_1: BUFFER_MUTATION target='bn2.num_batches_tracked'
            linear_1: USER_OUTPUT

        Range constraints: {}

)
```